Intrusion Detection Systems (IDSs):
Guide to Snort

Kate Eggenberg
Department of Computing Sciences, Jacksonville University
CS330: Networks and Wireless Communication
Dr. Reza Sarraf
November 18th, 2025

What is an IDS?

An Intrusion Detection System, or IDS, is a monitoring tool that doesn't just sniff packets and log them like a network analyzer, such as Wireshark, would. Instead, it automates network security management by applying rules to incoming traffic and alerts either a Security Information and Event Management system (SIEM) or security administrators of suspicious or malicious activity on the network (IBM, 2024). Organizations can use an IDS for identifying threats, logging network activity, and ensuring the security policy integrity of, say, an institution's firewall.

According to Scarfone & Mell, the primary functions of an IDS are the logging of information related to monitored events, the notification of security administrators of such events, and the production of reports that summarize the events of interest. There are Intrusion Prevention Systems (IPS) that are different from an IDS in that they can actually attempt to prevent threats from succeeding. Snort is an example of one of the original IDS/IPS software available, and it remains open-source to this day. This paper aims to examine the fundamentals of Intrusion Detection Systems and then introduce Snort while showcasing how to set rules and examples of them in action.

Types of IDS

According to IBM, there are four types of IDS, but for brevity's sake, this paper will cover the two most prominent types: Network Intrusion Detection Systems (NIDSs) and Host Intrusion Detection Systems (HIDSs).

NIDSs monitor copies of packets coming into the network and going out of the network. They can be placed at a variety of different locations in the network depending on their purpose. Putting it closer to the network perimeter allows it to detect incoming threats. Putting an NIDS deeper inside the network allows it to monitor copies of packets being passed between different subnets (IBM, 2024). Snort is an example of a NIDS.

HIDSs, as the name suggests, are a type of IDS that are kept locally on host devices and operate by taking snapshots of the system and its files periodically to monitor them. It is commonly used in tandem with a NIDS, and can serve as extra defense in case of ransomware on one host device (IBM, 2024).

Detection Methodologies

There are a few common detection methodologies used by Intrusion Detection Systems, including signature-based detection, anomaly-based detection, and stateful protocol analysis. Snort uses a combination of the three to detect threats.

For detecting known threats, signature-based detection is simple and effective at preventing threats that are already known to be malicious. This method uses signatures, or known patterns that are associated with known threats, in a list to compare to activity and events. If it finds matches between some characteristic or behavior of an event to one of its known signatures, it can flag this event. (Scarfone & Mell, 2007, sec. 2.3.1) An example of signature-based detection could be when an IDS flags an event where an email is received with the title, 'Flash Upgrade!', and it contains an attached file 'get_flash_upgrade.exe', where 'get_flash_upgrade.exe' is a malware signature on the IDS's list. While signature-based detection is effective for already known threats, it does not identify zero-day threats that have yet to have their signatures documented by lists that the IDS keeps updated.

This is where anomaly-based detection steps in, solving the issue of signature-based detection being unable to detect unknown attacks. Anomaly-based detection has a period of analysis that helps to develop a baseline profile for activity that is deemed normal or baseline, called a training period (Scarfone & Mell, 2007, sec. 2.3.2). With this profile, it can then statistically determine when activity from hosts, users, applications, etc, is within normal bounds or abnormal. So instead of relying on a list of known signatures associated with malicious behavior, it can take the baseline model it made and compare it to any abnormal behavior, like a user suddenly active outside of their normal hours in a different location, downloading gigabytes of data. A flaw seen with this approach is that the generated profile may incidentally include malicious behavior and therefore not flag it. Additionally, false positives are a common issue with anomaly-based detection, as activity that is benign but deviates from the profile can be flagged.

Finally, stateful protocol detection uses predefined examples or expected behavior for different activities, typically provided by an organization or institution, instead of developing its own profile of normal behavior. Similarly, it compares activity to this baseline and flags activity that deviates from the vendor's described normal (Scarfone & Mell, 2007, sec. 2.3.3).

What is Snort?

According to Snort.org's short installation and configuration how-to video, Snort is a free open-source Network Intrusion Detection and Prevention System created in 1998, aiming to streamline network monitoring and can be used on Linux or Windows operating systems. After its parent company was bought by Cisco in 2013, Snort expanded to use in commercial solutions like FirePower and Meraki. It uses rules to define malicious activity in a network and compares packets to those rules to generate warnings. It can also be deployed in line as an IPS. Snort has three main modes: Sniffer mode, Packet Logger mode, and Network Intrusion Detection System (NIDS) mode (Lenaerts-Bergmans, 2023). The sniffer mode takes packets from the network and displays them to the user, similar to Wireshark. Packet Logger mode, as the name suggests, logs packets sniffed from the network in storage of your choice. Finally, the NIDS mode is the customizable mode in which the monitoring and analysis of the network occur. (Cisco, 2020). For this paper's purposes, we will be using the NIDS mode.

Snort's Workflow

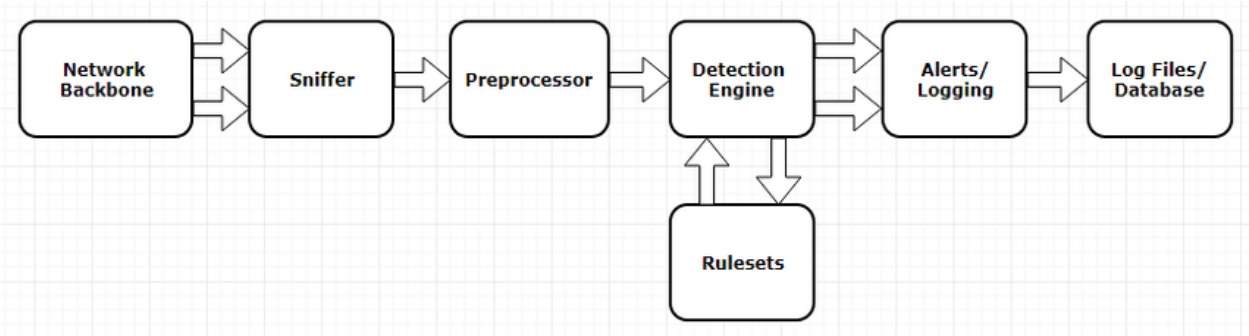

Figure 1 - Depiction of the components of the architecture within Snort (Biju, 2018).

Snort has four main components that make up its architecture. First, the Data Acquisition System (DAQ), or the packet sniffer module that gathers data. Second, the preprocessor helps prepare packets for inspection, such as reassembling fragmented packets so that the detection engine can evaluate full payloads. Third, the detection engine runs these packets against rulesets to determine suspicious or malicious activity. Fourth, Snort produces output in the form of alerts or logging in log files or a database, depending on the content of packets (Snort.org, 2015).

Snort Rules

Before creating working rules, Snort needs to be configured for your purposes. This includes setting your home network, where you want to store log files, and more. This paper will not go into detail on the configuration process, but one can read Snort's User Manual to gain further knowledge on the installation and configuration process.

Knowing that Snort can be used as a NIDS and has a detection engine that relies on rulesets, how do we set rules for Snort?

According to the Cisco Talos Detection Response Team's Snort 3 Rule Writing Guide, each rule within Snort has a header and then options. As seen in Figure 2, Snort's rules have a header and body. The header contains important fields like action, protocol, sourceIP, sourceport, destIP, and destport.

Figure 2 - Fields of Snort Rule header: action, protocol, source IP, source port, and Snort Rule body: rule options (Barrientos, 2022).

The action field is where you can dictate the action taken upon finding a match to the rule. If running Snort simply as an IDS, not an inline IPS, then the three default actions are alert, log, and pass. Alert generates an alert for the current packet, log logs the current packet, and pass marks the current packet as passed. The next field, protocol, dictates what protocols the rule should look at, including layer 3 ip and icmp and layer 4 tcp and udp. There is also functionality for writing application layer services in this rule field, like http, ssl, or smtp, where Snort should only try to match traffic that is this layer 5 service. The third and sixth fields both concern IP addresses, specifically source and destination. A rule must have the same source and destination IP addresses to match, and these IP addresses can be declared in a couple of ways: IP address with or without CIDR block, a variable previously defined in configuration representing a single or set of network addresses, keyword 'any' representing any IP address, and finally a list of IP addresses in square brackets separated by commas (Cisco Talos Detection Response Team, 2023). The fourth and seventh fields of the rule header are source and destination port numbers

and have similar declaration methods as IP addresses, while also including the ability to indicate port number ranges using a colon, like so, 1:1024 (indicates port numbers 1 to 1024). The last field in the rule header is the direction operator. This shows the direction of traffic that the rule applies to. There are two direction operators: -> and <>. -> indicates fields on the left side of the operator are source and fields on the right side of the operator are destination. While <> is the bidirectional operator indicating either side could be considered source and destination. Snort3 also provides functionality that streamlines rule creation, making the IP and port fields unnecessary for service and file rules (Cisco Talos Detection Response Team, 2023).

```
alert tcp $EXTERNAL_NET 80 -> $HOME_NET any
```

Figure 3 - Example Snort Rule Header (Created by author)

In Figure 3, we can see an example of a Snort rule header. Using what we learned about header fields, we can interpret that this rule header is making a rule that will monitor tcp protocol incoming from an external network source IP address on source port 80 to the home network destination IP address on any destination port and generate an alert for anything in these fields matching the omitted rule options.

Rule options are the instructions for analyzing packets. Figure 4 showcases what Snort rule options look like. Rule options are enclosed in parentheses. Typically, the rule name is to the left of a colon, and then there is optional criteria to the right of the colon. Each line ends with a semicolon (Cisco Talos Detection Response Team, 2023).

```
(
    msg:"Attack attempt!";
    flow:to_client,established;
    file_data;
    content:"1337 hackz 1337", fast_pattern, nocase;
    service:http;
    sid:1;
)
```

Figure 4 - Example Snort Rule Options (Cisco, 2020)

There are four major types of rule options according to Cisco's Talos Team Snort 3 Rule Writing Guide: general, payload, non-payload, and post-detection. This paper will cover the first three. General rules provide context for a rule. Some of the most important general rules include msg, which gives the message printed when a rule matches a packet; sid, which gives the signature number of a rule; rev, which gives the version number of a rule; and classtype, which classifies the type of attack the rule monitors and gives it priority. Payload detection rule options are what evaluate a packet's payload, and they guide Snort on what, where, and how to look for said payload data. For brevity's sake, this paper will touch on only a couple of the payload rules: the content rule option and a couple of http specific rule options. The content rule does pattern matching against data in the payload of packets. It can include ASCII strings, hex bytes, or both, and there can be multiple content matches that payload data can be compared to in one content rule (Cisco Talos Detection Response Team, 2023). Some important http specific options include http_uri, http_header, and http_method. Finally, non-payload detection rule options are rules that monitor all the other parts of a packet that aren't the payload. Some common rules are flags, seq, and ack.

Creating Rules

Now that this paper has outlined what Snort is, what IDSs are, what Snort rules are, and how to create them, we can make some of our own rules and show them in action in a test Snort implementation.

For my first example rule, I want to create a rule that detects suspicious port scanning activity, like someone attempting to scan TCP ports. To review, when someone wants to establish a connection to a TCP port, both sides must go through a three-way handshake. The first packet in the three-way handshake is flagged as SYN in the TCP packet header. This flag means that someone is trying to establish a connection with the receiving port. People using port scanning can use this to see if they get a response back to the SYN. If they do, this indicates a port is open. This behavior is often associated with malicious intentions. Thus, making a rule to detect it is crucial.

We can start by determining what action we wish to take upon the match of a packet to our rule. For this demo of Snort, let's use **alert** as our action. Next, we should determine which protocol we are monitoring. Because we want to identify when someone scans TCP ports, we

should use **tcp** as our protocol. Next, we should determine the source IP address and source port number we wish to monitor for this behavior. Since any external incoming packet exhibiting this behavior should be monitored, we can make sourceIP and sourceport both use the **any** keyword. Next, the destination IP address and port number. We want to monitor our network and our ports, so we should use our previously configured home network variable (**$HOME_NET**) and then the **any** keyword to monitor all ports. Next, we should use the **->** direction operator to indicate we want to monitor traffic coming from the external network to our network. Finally, we need to fill out our rule options. We know that a port scan can look like a SYN flag being present in the TCP packet headers. We can create a non-payload detection rule with **flags: S**. We need to add a **msg** option that describes what we are alerting for, **msg: "TCP SYN Flag Detected!"**. We need to add **sid** or the rule signature identifier. 1000000 and greater are reserved for user-created rules, so this rule can use **sid:1000000**. We should also indicate that this is the first version of this rule. We can use **rev:1**. Putting that all together, Figure 5, showcases the layout of our user-generated Snort rule.

```
alert tcp any any -> $HOME_NET any
(
     flags: S;
     msg: "TCP SYN Flag Detected!";
     sid: 1000000;
     rev: 1;
)
```

Figure 5 - TCP Port Scanning Snort Rule (Created by Author)

```
log tcp any any -> $HOME_NET 80 (msg: "Attempt to access
malicious file."; content: "malware.zip"; sid: 1000001; rev: 1;)
```

Figure 6 - Suspicious Payload Data Snort Rule (Created by Author)

For our second user-generated rule, we can focus on suspicious HTTP packets. Snort should detect when an incoming TCP HTTP packet is carrying the ASCII text: "malware.zip" in its payload and log this packet. Figure 6 showcases what this Snort Rule would look like.

Testing Snort

What does it look like when Snort applies these rules and finds a match? For this paper, I set up Snort in a Kali Linux Virtual Machine, making sure it was on the host-only network adapter instead of NAT. To start running Snort in NIDS mode, in a terminal in Linux, you can use the following command: snort -i 3 -c C:\Snort\etc\snort.conf -A console. This chooses which interface Snort is listening to and tells it to put alerts to the console (Cisco Talos Detection Response Team, 2023). I configured my Snort.lua file to make sure the Kali Linux host-only IP address was set as the HOME_NET variable. Additionally, I created a local.rules file containing my two user-created rules and added that to the configuration. Next, I opened up port 80 in the Kali terminal, so when testing the suspicious payload data snort rule, I had an open HTTP port to communicate with. By downloading nmap on my Windows OS and using it to scan the HOME_NET address on Kali I was able to generate alerts on the terminal running Snort on my Kali Linux Virtual Machine. Next, I tried echoing “malware.zip” to port 80 on my Kali Linux Virtual Machine in my Windows terminal. This succeeded in making Snort log the packet.

```
{TCP} 192.168.56.1:53412 -> 192.168.56.101:80
11/18-21:47:32.912345  [**] [1:1000000:1] TCP SYN Flag Detected [**]
[Priority: 0]
```

Figure 7 - Alert triggered when the rule created in Figure 5 finds a match. (Created by author)

Conclusion

Snort is the basis for many commercial IDS/IPSs and has laid the backend process for most modern network monitoring. By maintaining its open-source origins even after becoming a part of Cisco, it allows users to have access to this crucial network security tool. Using Snort with a combination of multiple different detection methods is the most secure approach. While administrators can sit back and create custom rules all day, it clearly is not as efficient as relying on predetermined rule sets that are updated with every new exploit and threat recognized. In addition, anomaly-based detection allows us to be more prepared for the unknown. New zero-days have the possibility to wreak havoc on a network, and by developing an image of what is normal, we can hopefully detect them and prevent more damage. Finally, stateful-based protocols once again save the administrator and Snort time in generating a baseline.

References

Barrientos, C. (2022, October 26). *Understand snort3 rules*. Cisco. https://www.cisco.com/c/en/us/support/docs/security/ios-intrusion-prevention-system-ips/218349-understand-snort3-rules.html

Biju, I. (2018). *Intelligent Intrusion Detection System Through Combined and Optimized Machine Learning* - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/The-Snort-architecture_fig1_325851146

Cisco. (2020). *SNORT Users Manual 2.9.16.* Amazonaws.com http://manual-snort-org.s3-website-us-east-1.amazonaws.com/ Cisco Talos Detection Response Team. (2023). *Snort 3 rule writing guide*. Snort 3 Rule Writing Guide - Snort 3 Rule Writing Guide. https://docs.snort.org/welcome

Lenaerts-Bergmans, B. (2023, April 24). *Snort explained: Understanding snort rules and use cases*. CrowdStrike. https://www.crowdstrike.com/en-us/cybersecurity-101/threat-intelligence/snort-rules/#:~:text=Real%2Dtime%20network%20traffic%20monitoring,Compatibility%20with%20any%20OS

IBM. (2024). *What is an intrusion detection system (IDS)?* IBM. from https://www.ibm.com/think/topics/intrusion-dectection-system

Scarfone, K., & Mell, P. (2007). *Guide to intrusion detection and prevention systems (IDPS)* (NIST Special Publication 800-94). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-94.pdf

Snort3. (2025). *Snort++* (Version 3.9.7.0) [Computer Software]. GitHub. Retrieved from https://github.com/snort3/snort3

Snort.org. (2015). *Multimedia Snort How-To's: Snort Installation and Configuration*. Snort. https://snort-org-site.s3.amazonaws.com/production/document_files/files/000/000/125/original/Techbyte_Snort_Final_v2_1080p.mp4?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIAU7AK5ITMMFKW2CPY%2F20251117%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20251117T205428Z&X-Amz-Expires=172800&X-A

mz-SignedHeaders=host&X-Amz-Signature=3ede7ff076dc36ff8f58830a629693d4e8081365d00be0e047be9829c1fa9573

PowerPoint used Designer to format slides, fonts, and transitions